Practical Introduction to High Performance Computing

Parallel Performance and Scalability
(Preview: Only Pages 1 to 3)

H. Noah Belzer — 5 January 2026

The University of Edinburgh

School of Informatics

EPCC

Table of Contents

Introduction **3**

1 Experiment 1: Investigating Scalability **4**
1.1 Tables of Results . . . 5
1.2 Speedup and Efficiency Plots . . . 6
1.3 Comparison to Serial Result . . . 6

2 Experiment 2: Investigating the Effect of Additional Reynolds Number (Re) Computations **7**
2.1 Tables of Results . . . 7
2.2 Speedup and Efficiency Plots . . . 8
2.3 Comparison to Serial Result . . . 9

3 Experiment 3: How Performance Varies with Choice of Compiler and Compiler Options **9**
3.1 Tables of Results . . . 10
3.2 Cell Update Rate vs. Runtime Plots . . . 11
3.3 Comparison to Serial Result . . . 13

4 Experiment 4: Comparing Performance Differences for the C and Fortran Versions of the Code **13**
4.1 Tables of Results . . . 13
4.2 Speedup and Efficiency Plots . . . 14
4.3 Comparison to Serial Result . . . 15

Conclusion **15**

Introduction

Problem Statement
The goal of this report is to perform four experiments using the provided parallel computational fluid dynamics (CFD) code, document the results, and analyse the findings to draw conclusions. The four experiments include the investigation of scalability for different problem sizes, investigating the effect of additional computations from Reynolds number (Re), observing how the performance varies with choice of compiler and compiler options, and comparing the C and Fortran versions of the parallel CFD code. These investigations will reveal the differences in performance metrics between the serial and parallel versions, and we can see how different scale factors, numbers of iterations, compilers, and compiler flags influence the overall performance using varying numbers of nodes.

Note: All graphs in this report were created using MATLAB. We exported the graphs using a PDF format and inserted them as figures. We used the ARCHER2 HPC Cluster to run all the code in this report.

Summary of the Problem
The problem we are addressing in this report is determining the flow pattern of a fluid in a cavity. We have a square box with an inlet on the right side of the square and an outlet on the bottom side of the square. The problem is in two dimensions, and we assume that there is zero viscosity.[1]

Mathematically, at zero viscosity, the stream function, Ψ, satisfies

$$\nabla^2\Psi = \frac{\partial^2\Psi}{\partial x^2} + \frac{\partial^2\Psi}{\partial y^2} = 0. \tag{1}$$

We use the finite difference form:[1]

$$\Psi_{i-1,j} + \Psi_{i+1,j} + \Psi_{i.j-1} + \Psi_{i,j+1} - 4\Psi_{i,j} = 0. \tag{2}$$

To find solutions, we will use the Jacobi iterative method.[1] We repeat this process for many iterations:

```
Loop over all points i and j:

    psinew[i][j] = 0.25 * ( psi[i+1][j] + psi[i-1][j] +
                            psi[i][j+1] + psi[i][j-1]   )

Copy psinew back to psi for the next iteration.
```

Since the boundary values are fixed, the stream function, Ψ, can be calculated for each point by averaging the value at that point with its four nearest neighbours.[1] This process continues until the algorithm converges on a solution, which stays unchanged by the averaging. In terms of parallelism, the algorithm calculates the value at each grid point by combining it with the value of its neighbours. The same amount of work is needed to calculate each grid point. The grid is broken up into smaller grids, and one is allocated to each process. The program utilises halo swapping to handle points on the edge of the grid.[1] For our experiments, we used a fixed number of iterations across all problem sizes to ensure comparable scalability and performance values, so the trials may stop before convergence is reached. Using a fixed number of iterations is appropriate for performance measurement.[1]

Description of the Parallel CFD Code
(Hidden)

[1]EPCC, The University of Edinburgh, *Computational Fluid Dynamics*, slides 4–12.